Ensembles

10

Post

CS 3244
Machine Learning

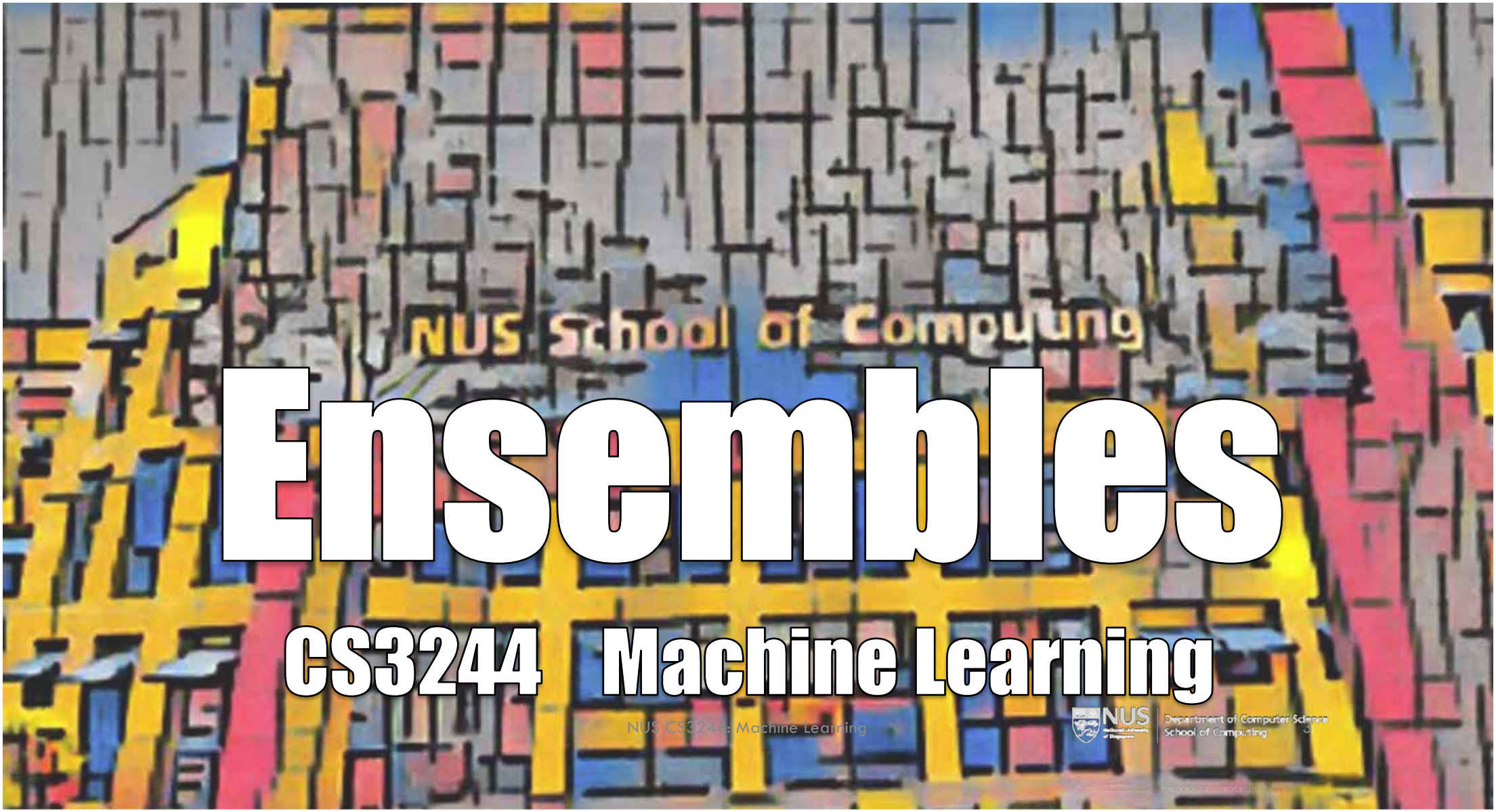
NUS | Computing
National University of Singapore

Recap and Forecast

- Understand why deep learning was enabled by certain key technological milestones;
- Understand the Recurrent and Convolutional NN architectures (shortened as RNNs and CNNs);
- Understand how DNNs regularize and optimize via dropout, pretraining and choice of activation function;
- Relate embeddings as a method to incorporate prior knowledge as a hidden layer;
- Obtain practical experience in constructing DNNs with PyTorch.

Learning Outcomes for this week:

- Understand & implement Decision Trees;
  - Master Information Gain as a means of choosing optimal attributes for decisions;
- Describe ensembling methods that integrate multiple classifiers;
- Examine Boosting and Bagging as basic techniques for extending ensembles;
  - Relate AdaBoost and Random Forests as extensions of Decision Trees using both techniques.



Ensembles

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Ensembles: Revisiting Netflix

Just three weeks after it began, at least 40 teams had bested the Netflix classifier.

Top teams showed about 5% improvement.

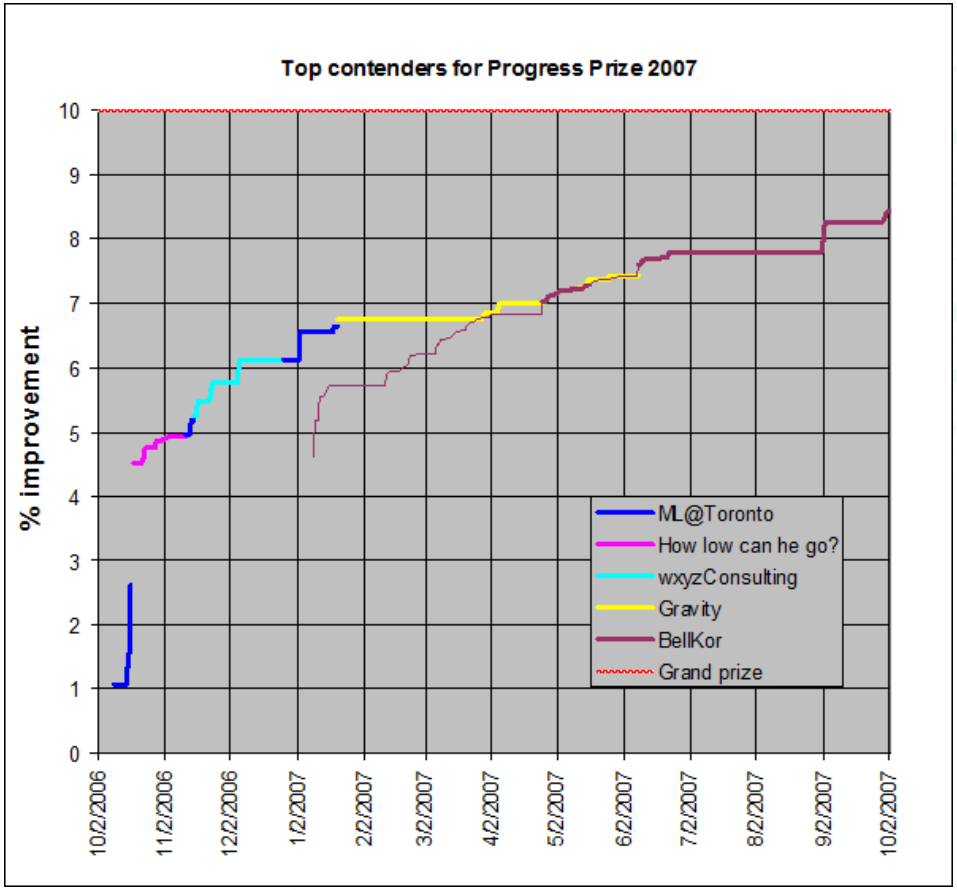
Netflix Prize

me | Rules | Leaderboard | Register | Update | Submit | Download

Leaderboard

Team Name	Best Score	% Improvement
No Grand Prize candidates yet	--	--
Grand Prize - RMSE <= 0.8563		
How low can he go?	0.9046	4.92
ML@UToronto A	0.9046	4.92
ssorkin	0.9089	4.47
wxyzconsulting.com	0.9103	4.32
The Thought Gang	0.9113	4.21
NIPS Reject	0.9118	4.16
simonfunk	0.9145	3.88
Bozo_The_Clown	0.9177	3.54
Elliptic Chaos	0.9179	3.52
datcracker	0.9183	3.48
Foreseer	0.9214	3.15
bsdfish	0.9229	3.00
Three Blind Mice	0.9234	2.94
Bocsimacko	0.9238	2.90
Remco	0.9252	2.75
karmatics	0.9301	2.24
Chapelator	0.9314	2.10
Flmod	0.9325	1.99
mthrox	0.9328	1.96

However, improvement slowed…



Source:
http://www.research.att.com/~volinsky/netflix/

Rookies

"Thanks to Paul Harrison's collaboration, a simple mix of our solutions improved our result from 6.31 to 6.75"

--	No Progress Prize candidates yet	--	--
Progress Prize - RMSE <= 0.8625			
1	BellKor	0.8705	8.50
Progress Prize 2007 - RMSE = 0.8712 - Winning Team: KorBell			
2	KorBell	0.8712	8.43
3	When Gravity and Dinosaurs Unite	0.8717	8.38
4	Gravity	0.8743	8.10
5	basho	0.8746	8.07
6	Dinosaur Planet	0.8753	8.00
7	ML@UToronto A	0.8787	7.64
8	Arek Paterek	0.8789	7.62
9	NIPS Reject	0.8808	7.42
10	Just a guy in a garage	0.8834	7.15
11	Ensemble Experts	0.8841	7.07
12	mathematical capital	0.8844	7.04
13	HowLowCanHeGo2	0.8847	7.01
14	The Thought Gang	0.8849	6.99
15	Reel Ingenuity	0.8855	6.93
16	strudeltamale	0.8859	6.88
17	NIPS Submission	0.8861	6.86
18	Three Blind Mice	0.8869	6.78
19	TrainOnTest	0.8869	6.78
20	Geoff Dean	0.8869	6.78
21	Rookies	0.8872	6.75
22	Paul Harrison	0.8872	6.75
23	ATTEAM	0.8873	6.74
24	wxyzconsulting.com	0.8874	6.73
25	ICMLsubmission	0.8875	6.72
26	Efratko	0.8877	6.70
27	Kitty	0.8881	6.65
28	SecondaryResults	0.8884	6.62
29	Birgit Kraft	0.8885	6.61

Arek Paterek

"My approach is to **combine the results of many methods** (also two-way interactions between them) using linear regression on the test set. The best method in my ensemble is regularized SVD with biases, post processed with kernel ridge regression"

http://rainbow.mimuw.edu.pl/~ap/ap_kdd.pdf

--	No Progress Prize candidates yet	--	--
Progress Prize - RMSE <= 0.8625			
1	BellKor	0.8705	8.50
Progress Prize 2007 - RMSE = 0.8712 - Winning Team: KorBell			
2	KorBell	0.8712	8.43
3	When Gravity and Dinosaurs Unite	0.8717	8.38
4	Gravity	0.8743	8.10
5	basho	0.8746	8.07
6	Dinosaur Planet	0.8753	8.00
7	ML@UToronto A	0.8787	7.64
8	Arek Paterek	0.8789	7.62
9	NIPS Reject	0.8808	7.42
10	Just a guy in a garage	0.8834	7.15
11	Ensemble Experts	0.8841	7.07
12	mathematical capital	0.8844	7.04
13	HowLowCanHeGo2	0.8847	7.01
14	The Thought Gang	0.8849	6.99
15	Reel Ingenuity	0.8855	6.93
16	strudeltamale	0.8859	6.88
17	NIPS Submission	0.8861	6.86
18	Three Blind Mice	0.8869	6.78
19	TrainOnTest	0.8869	6.78
20	Geoff Dean	0.8869	6.78
21	Rookies	0.8872	6.75
22	Paul Harrison	0.8872	6.75
23	ATTEAM	0.8873	6.74
24	wxyzconsulting.com	0.8874	6.73
25	ICMLsubmission	0.8875	6.72
26	Efratko	0.8877	6.70
27	Kitty	0.8881	6.65
28	SecondaryResults	0.8884	6.62
29	Birgit Kraft	0.8885	6.61

When Gravity and Dinosaurs Unite

"Our common team blends the result of team Gravity and team Dinosaur Planet."

Might have guessed from the name…

--	No Progress Prize candidates yet	--	--
Progress Prize - RMSE <= 0.8625			
1	BellKor	0.8705	8.50
Progress Prize 2007 - RMSE = 0.8712 - Winning Team: KorBell			
2	KorBell	0.8712	8.43
3	When Gravity and Dinosaurs Unite	0.8717	8.38
4	Gravity	0.8743	8.10
5	basho	0.8746	8.07
6	Dinosaur Planet	0.8753	8.00
7	ML@UToronto A	0.8787	7.64
8	Arek Paterek	0.8789	7.62
9	NIPS Reject	0.8808	7.42
10	Just a guy in a garage	0.8834	7.15
11	Ensemble Experts	0.8841	7.07
12	mathematical capital	0.8844	7.04
13	HowLowCanHeGo2	0.8847	7.01
14	The Thought Gang	0.8849	6.99
15	Reel Ingenuity	0.8855	6.93
16	strudeltamale	0.8859	6.88
17	NIPS Submission	0.8861	6.86
18	Three Blind Mice	0.8869	6.78
19	TrainOnTest	0.8869	6.78
20	Geoff Dean	0.8869	6.78
21	Rookies	0.8872	6.75
22	Paul Harrison	0.8872	6.75
23	ATTEAM	0.8873	6.74
24	wxyzconsulting.com	0.8874	6.73
25	ICMLsubmission	0.8875	6.72
26	Efratko	0.8877	6.70
27	Kitty	0.8881	6.65
28	SecondaryResults	0.8884	6.62
29	Birgit Kraft	0.8885	6.61

Ensembles, for the win

The team BellKor's Pragmatic Chaos is a combined team of BellKor, Pragmatic Theory and BigChaos. BellKor consists of Robert Bell, Yehuda Koren and Chris Volinsky. The members of Pragmatic Theory are Martin Piotte and Martin Chabbert. Andreas Töscher and Michael Jahrer form the team BigChaos.

"…There were two main factors which improved the overall accuracy: The quality of the individual algorithms and the ensemble idea … Best blending results are achieved when the whole ensemble has the right tradeoff between diversity and accuracy. "

Leaderboard

Display top 20 leaders.

Rank	Team Name	Best Score	% Improvement	Last Submit Time
1	BellKor's Pragmatic Chaos	0.8558	10.05	2009-06-26 18:42:37
Grand Prize - RMSE <= 0.8563				
2	PragmaticTheory	0.8582	9.80	2009-06-25 22:15:51
3	BellKor in BigChaos	0.8590	9.71	2009-05-13 08:14:09
4	Grand Prize Team	0.8593	9.68	2009-06-12 08:20:24
5	Dace	0.8604	9.56	2009-04-22 05:57:03
6	BigChaos	0.8613	9.47	2009-06-23 23:06:52

http://www.netflixprize.com/assets/GrandPrize2009_BPC_BigChaos.pdf

Stock Market – Ensemble

We have T reports $h_1, h_2, \ldots, h_T$ predicting whether a stock will go up as $h(x)$.

We can:

1. Select the most trustworthy of them based on their usual performance
   Validation: $h(x) = h_{t^*}(x)$ with $t^* = argmin_{t \in \{1,2,\ldots,T\}} J_{val}(h_t^-)$

2. Let each report have a vote
   Uniform Vote: $h(x) = \text{sign}\left(\sum_{t=1}^{T} 1 \cdot h_t(x)\right)$

3. Weight the reports non-uniformly
   Weighted Vote: $h(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$ where $\alpha_t \geq 0$.

4. Combine the predictions conditionally
   We just finished this – Decision Trees! Base learner a *decision stump*

Key: component hypotheses are diverse

Blending models: aggregating after obtaining $h_1, h_2, \ldots, h_T$.

What about obtaining $h(x)$ during the learning process?

Bootstrap & Bagging

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Diversifying the Data Set

We'd like diverse hypotheses by training on diverse sets of data, one for each h_t. Recall validation's h^-.

Can we manufacture more data than observed in our m samples?

Bootstrapping: a statistical tool that re-samples from $\mathcal{D}$ to simulate $\mathcal{D}_t$.

Similar in spirit to dataset augmentation, common in deep learning.

Bagging: Bootstrap AGgregation

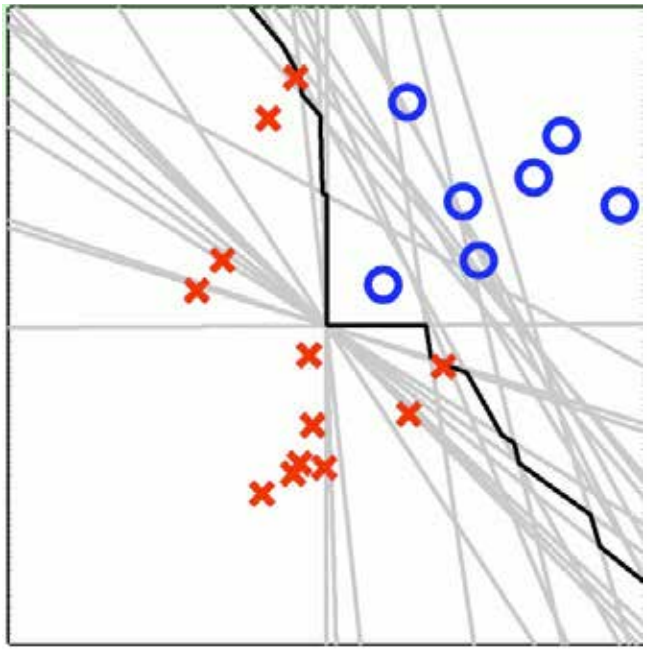
Bootstrap sample $\widetilde{\mathcal{D}}_t$: re-sample m examples from $\mathcal{D}$ uniformly with replacement, for data randomness.

Bootstrap Aggregation:

For $t = 1,2,\ldots,T$:
    Request m sized data $\widetilde{\mathcal{D}}_t$ from bootstrapping
    Obtain h_t by $\mathcal{A}(\widetilde{\mathcal{D}}_t)$
$h(x) = \mathrm{Uniform}(\{h_t\})$

A simple meta algorithm on top of base algorithm $\mathcal{A}$.

PLA, bagged



$T_{BAG} = 25$

Diverse h_t from bagging

Non-linear boundary after aggregation

Bagging works reasonably if $\mathcal{A}$ is sensitive to data randomness.

63.2%

For large m, the set $\widetilde{\mathcal{D}}_t$ is expected to have the fraction $1 - \left(1 - \frac{1}{m}\right)^m \approx 1 - \frac{1}{e} = 63.2\%$ of the unique examples in $\mathcal{D}$; the rest are duplicates.

This means each h_t is trained with significantly less data

The final model $h(x) = \mathrm{Uniform}(\{h_t\})$ has seen all the data points, for large m.

On average, an example will be in the majority of the bootstrap samples $\widetilde{\mathcal{D}}_t$

Out of Bag

OOB	h_1	h_2	h_3	…	h_T
$(x^{(1)}, y^{(1)})$	$\widetilde{\mathcal{D}}_1$	*	$\widetilde{\mathcal{D}}_3$		$\widetilde{\mathcal{D}}_T$
$(x^{(2)}, y^{(2)})$	*	*	$\widetilde{\mathcal{D}}_3$		$\widetilde{\mathcal{D}}_T$
$(x^{(3)}, y^{(3)})$	*	$\widetilde{\mathcal{D}}_2$	*		$\widetilde{\mathcal{D}}_T$
…					
$(x^{(m)}, y^{(m)})$	$\widetilde{\mathcal{D}}_1$	$\widetilde{\mathcal{D}}_2$	*		*

* in the tth column: not used for obtained h_t. Called out-of-bag (OOB) examples for h_t. OOB size per $h_t = \mathbb{E}\left[\left(1 - \frac{1}{m}\right)^m\right] \approx 36.7\%$.

Use * to validate $h(x)$. $J_{oob}(h) = \frac{1}{m}\sum_{j=1}^{m} j(y^{(j)}, h_j^-(x^{(j)}))$, with h_j^- containing trees where $x^{(j)}$ is OOB.

OOB versus Validation

OOB	h_1	h_2	h_3	…	h_T
$(x^{(1)}, y^{(1)})$	$\widetilde{\mathcal{D}}_1$	*	$\widetilde{\mathcal{D}}_3$		$\widetilde{\mathcal{D}}_T$
$(x^{(2)}, y^{(2)})$	*	*	$\widetilde{\mathcal{D}}_3$		$\widetilde{\mathcal{D}}_T$
$(x^{(3)}, y^{(3)})$	*	$\widetilde{\mathcal{D}}_2$	*		$\widetilde{\mathcal{D}}_T$
…					
$(x^{(m)}, y^{(m)})$	$\widetilde{\mathcal{D}}_1$	$\widetilde{\mathcal{D}}_2$	*		*

Validation	h_1^-	h_2^-	h_3^-	…	h_T^-
$(x^{(1)}, y^{(1)})$	$\mathcal{D}_{train}$	$\mathcal{D}_{train}$	$\mathcal{D}_{train}$		$\mathcal{D}_{train}$
$(x^{(2)}, y^{(2)})$	$\mathcal{D}_{val}$	$\mathcal{D}_{val}$	$\mathcal{D}_{val}$		$\mathcal{D}_{val}$
$(x^{(3)}, y^{(3)})$	$\mathcal{D}_{val}$	$\mathcal{D}_{val}$	$\mathcal{D}_{val}$		$\mathcal{D}_{val}$
…					
$(x^{(m)}, y^{(m)})$	$\mathcal{D}_{train}$	$\mathcal{D}_{train}$	$\mathcal{D}_{train}$		$\mathcal{D}_{train}$

Bootstrapping as a Re-weighting process

$\mathcal{D} = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}), (x^{(4)}, y^{(4)})\}$

Via bootstrap: $\widetilde{\mathcal{D}}_t = \{(x^{(1)}, y^{(1)}), (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(4)}, y^{(4)})\}$

Weighted J_{train} on $\mathcal{D}$

$$J^{u}_{train}(h) = \frac{1}{4}\sum_{j=1}^{4} u_j^{(t)} \cdot [[y^{(j)} \neq h(x^{(j)})]]$$

$(x^{(1)}, y^{(1)}), u_1 = 2$
$(x^{(2)}, y^{(2)}), u_2 = 1$
$(x^{(3)}, y^{(3)}), u_3 = 0$
$(x^{(4)}, y^{(4)}), u_4 = 1$

J_{train} on $\widetilde{\mathcal{D}}_t$

$$J^{0/1}_{train}(h) = \frac{1}{4}\sum_{(x,y)\in\widetilde{\mathcal{D}}_t}^{4} [[y^{(j)} \neq h(x^{(j)})]]$$

$(x^{(1)}, y^{(1)})$ $(x^{(1)}, y^{(1)})$
$(x^{(2)}, y^{(2)})$

$(x^{(4)}, y^{(4)})$

Set of h_T is diverse by minimizing bootstrap weighted error.

Bagging reduces variance

Bagging reduces variance by averaging over many h_t trained over different $\widetilde{\mathcal{D}}_t$.

But has little effect on bias; still working with the whole spectrum of $\mathcal{H}$.

Can we use more simplified $\mathcal{H}$ for learning in ensembles?

Boosting & Adaboost

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Diversity over iterations: Boosting

How can we improve upon bagging?

Pay attention to which examples from the previous h_{t-1} which we get right and which we get wrong:

- Increase the weight of examples we get wrong
- Decrease the weight of examples we get right

Boosting: an ensemble meta-algorithm to create strong learners from weak (slightly better than random) learners.

- Reduces bias and variance, when weak learners are simple.

Adaboost: Adaptive Boosting

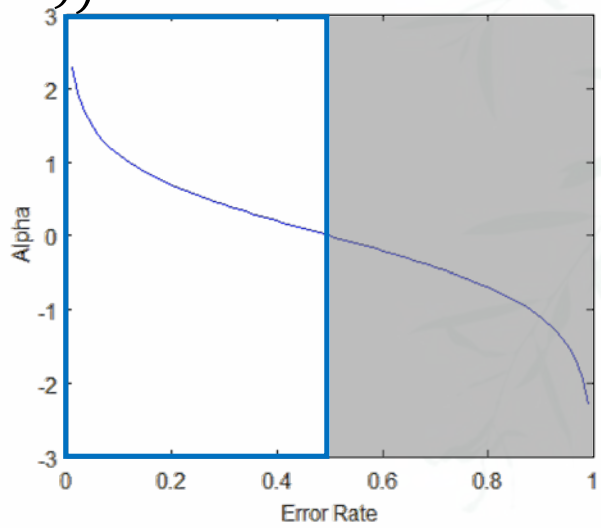
Let's see this at work in binary classification.

Weighted ensemble $h\left(x^{(j)}\right) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t\left(x^{(j)}\right)\right)$

Each classifier has weight α_t:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)^*,$$

where ϵ_t is the misclassification rate.



*α_t's form can be shown to be the minimizer of the exponential error function.

Adaboost: Reassigning the weights for examples

$$\mathcal{D}_{t+1}(j) = \frac{\mathcal{D}_t(j)\exp(-\alpha_t y^{(j)} \cdot h_t\left(x^{(j)}\right))}{Z_t}$$

Adaboost: Reassigning the weights for examples

$$\mathcal{D}_{t+1}(j) = \frac{\mathcal{D}_t(j)\exp(-\alpha_t y^{(j)} \cdot h_t\left(x^{(j)}\right))}{Z_t}$$

$y^{(j)} \cdot h_t\left(x^{(j)}\right) > 0$ if prediction is correct; weight shrinks exponentially
$y^{(j)} \cdot h_t\left(x^{(j)}\right) < 0$ if prediction is incorrect; weight grows exponentially

α_t's inclusion factors in the strength of the classifier.

Z_t is a normalizer to make weights for all training examples m sum to unity; each $\mathcal{D}_*$ can be thought of as a probability distribution.

A choice weak learner: Decision Stumps

a.k.a. 1-level decision tree:

$$h_{s,i,\theta}(x) = s \cdot \mathrm{sign}(x_i - \theta)$$

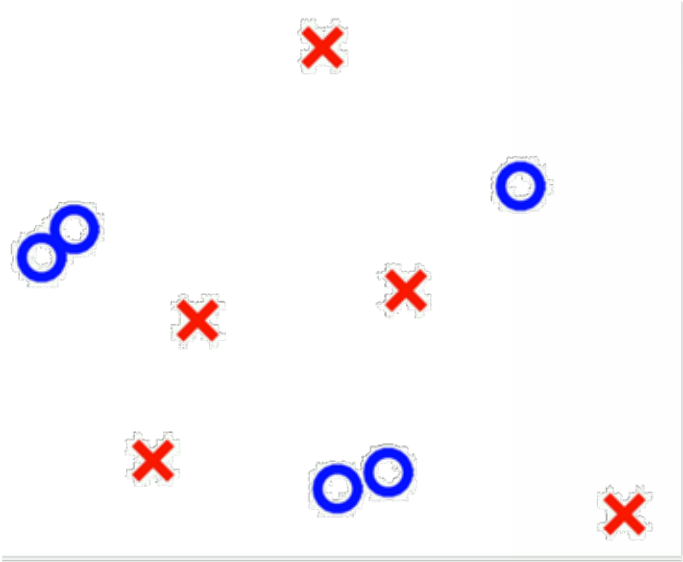
Positive and negative rays on some feature, three parameters (feature i, threshold θ, direction s)

Visualization: axis-perpendicular planes in $\mathbb{R}^n$.

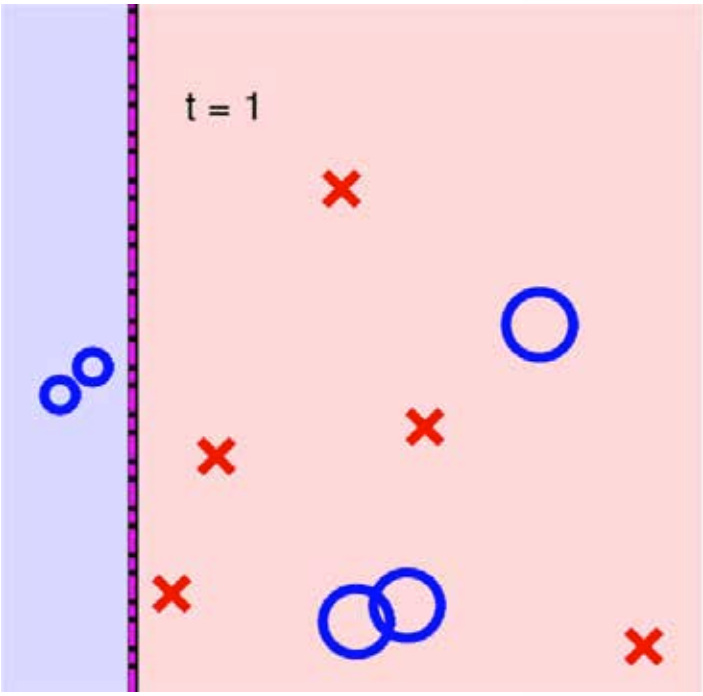
Efficient to optimize: $O(n \cdot \log m)$ time

Allows efficient minimization of J_{train}, but can be too weak to work by itself.

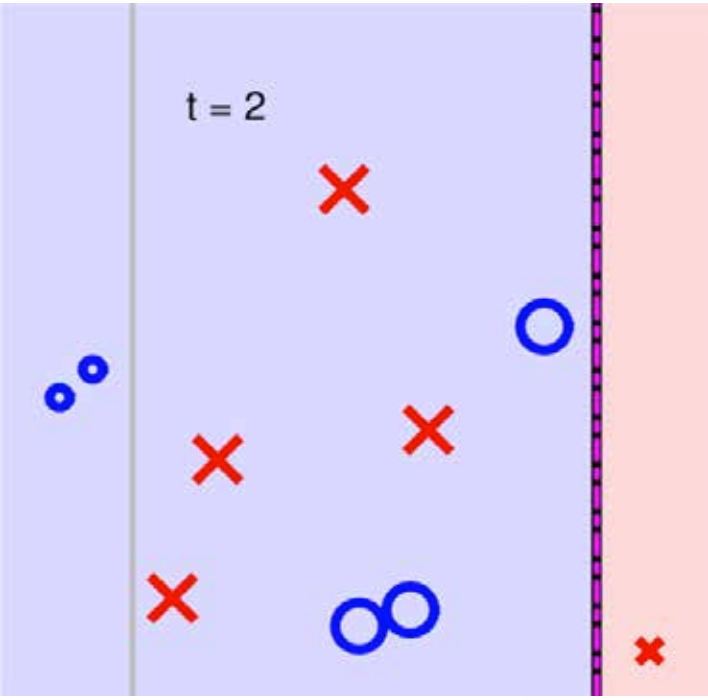
Adaboost–Stump



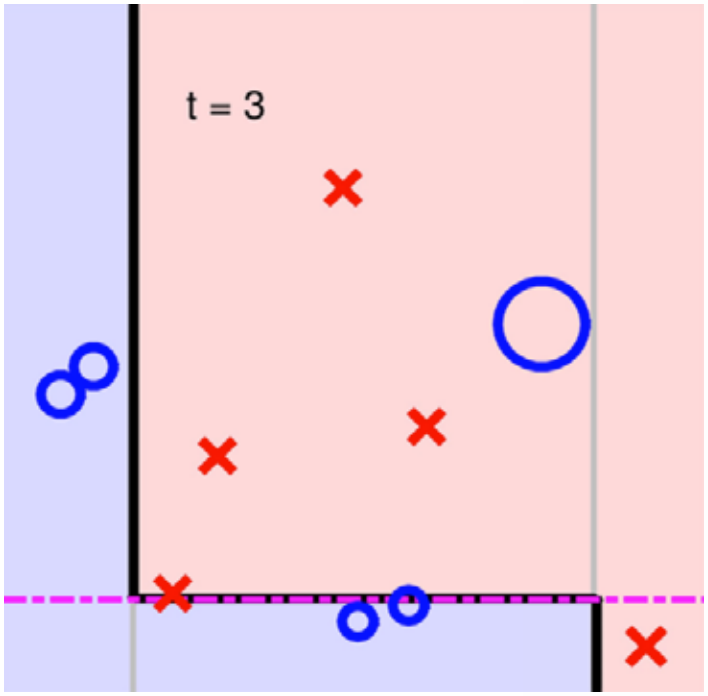
Adaboost – Decision Stump



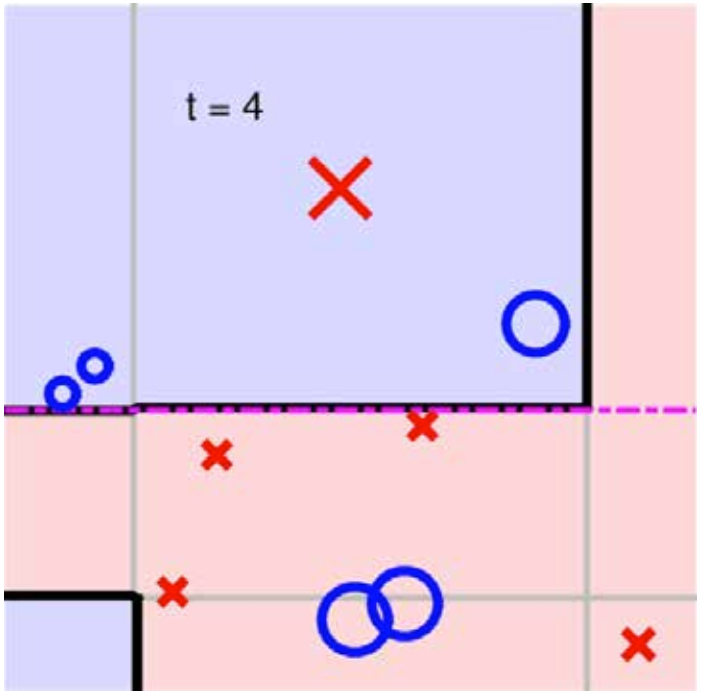
Adaboost – Decision Stump



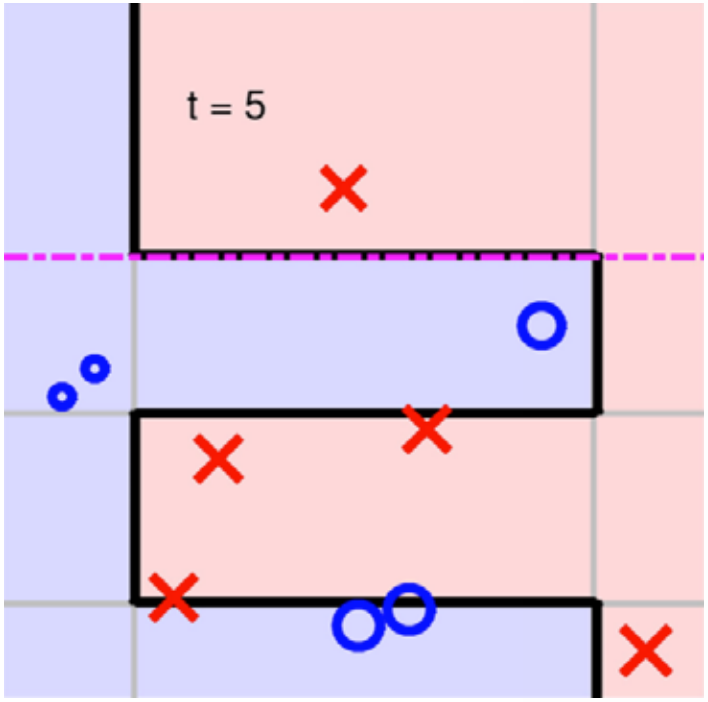
Adaboost – Decision Stump



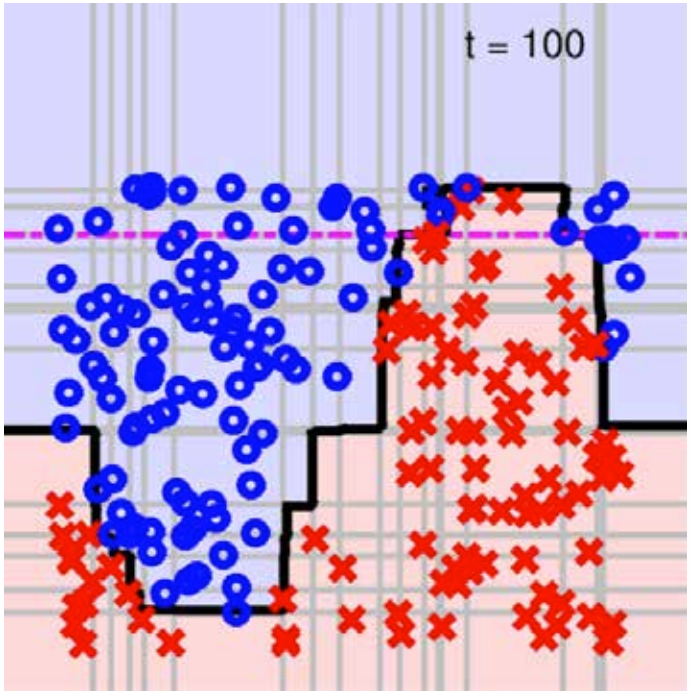
Adaboost – Decision Stump



Adaboost – Decision Stump



Adaboost – Decision Stump



Random Forest

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

So far…

Bagging

Bag($\mathcal{D}, \mathcal{A}$):

For $t = 1,2, \dots, T$:

Sample $\widetilde{\mathcal{D}}_t$ via bootstrapping with $\mathcal{D}$.

Obtain base h_t by $\mathcal{A}(\widetilde{\mathcal{D}}_t)$

Return $h(x) = \text{uniform}(\{h_t\})$

Reduces variance by voting/averaging.

Decision Tree

DTL($\mathcal{D}$):

If *termination* return base h_t, else

Learn Choose_Attribute(x), split $\mathcal{D}$ into $\mathcal{D}_c$.

Build h_c via DTL ($\mathcal{D}_c$) recursively

Return $h(x) =$

$$\sum_{c=1}^{C} [[\text{C_A}(x) = c]] h_c(x)$$

Large variance, especially if fully grown (unpruned).

Can we get the best of both worlds? Aggregate aggregations?

Random Forests

RandomForest($\mathcal{D}$):

For $t = 1,2,\ldots,T$:

Sample $\widetilde{\mathcal{D}}_t$ via bootstrapping with $\mathcal{D}$.

Obtain tree h_t by DTL$(\widetilde{\mathcal{D}}_t)$

Return $h(x) = \text{uniform}(\{h_t\})$

Highly parallel / efficient to learn

Inherits pros of DTL

Lessens overfitting of fully-grown trees.

But, RFs are not just bagging with DTL…

Decision Tree

DTL($\mathcal{D}$):

If *termination* return base h_t, else

Learn Choose_Attribute(x), split $\mathcal{D}$ into $\mathcal{D}_c$.

Build h_c via DTL $(\mathcal{D}_c)$ recursively

Return $h(x) =$

$$\sum_{c=1}^{C}\left[[\text{C_A}(x) = c]\right]h_c(x)$$

Diversifying by Feature Projection

Recall: data randomness for diversity in bagging:
Randomly sample m examples from $\mathcal{D}$.

How about another source of diversity, in terms of features?
Randomly sample n' features from x.

Often $n' \ll n$, useful for large n, such as $n' \equiv \sqrt{n}$

Random Forest re-samples new feature subspaces for each tree h_t*.

* Original RF algorithm does this per `Choose_Attribute` test.

Intuition behind random subspaces

Bagging can produce multiple h_t that are correlated. Why?

Also called *attribute* or *feature bagging*

The salient, high-IG features are likely to be the same over different bootstrapped samples.

When we sample n' features, we attend DTL to just that subset of features. Result: more diverse hypotheses h_t.

Works with other classifiers, not just decision trees in random forest.

Summary

Decision Trees

Expressive: can overfit to achieve $J_{train} = 0$
Need pruning to combat variance

Bootstrap & Bagging

Sample with replacement to generate new $\widetilde{\mathcal{D}}_t$.
With each train a h_t, apply uniform voting.
Variance lowered through averaging

Boosting

Make a strong classifier from simple weak classifiers.

Reweight $\widetilde{\mathcal{D}}_t$ to account for easy / hard examples.

Lower bias via simple hypotheses.

Random Forest

Combine Bagging with DTL.

Introduce more randomness through feature projection.

Each h_t tuned to some subset of the features.

Bottom Line: Diversity is good